

Project 2 - ECE1774

Project Overview

This project involves building an advanced power system simulator in Python to model and analyze a 7-node looped transmission system. The simulator supports power flow analysis, fault studies, and network configuration.

System Components

Circuit Class

- Manages the entire power system network.
- Stores system components in dictionaries for quick access.
- Provides methods to add buses, transformers, and transmission lines.
- Prepares the network for power flow analysis.

Bus Class

- Represents a node in the power system.
- Stores bus name and base voltage (kV).
- Tracks bus indexing for system-wide references.

Transformer Class

- Models a two-winding transformer.
- Connects two buses and computes impedance, admittance, and primitive Y-matrix.
- Supports per-unit calculations.

Transmission Line Class

- Models a transmission line connecting two buses.
- Computes series impedance, shunt admittance, and Y-matrix.
- Uses Conductor, Bundle, and Geometry as subclasses.

Settings Class

- Defines important global variables for other classes to use
- Has the frequency and base power for the system
- Base power has 2 values, both Mw and W, due to how the functions were coded

Solution Class

- Function for solving the Newton-Raphson method
- Takes the circuit class as the primary input, and others as needed (need more for this)

Conductor, Bundle, and Geometry Subclasses

- Define physical & electrical parameters of transmission lines.
- Conductor Class: Models a single conductor's electrical properties.
- Bundle Class: Manages multiple conductors per phase.
- Geometry Class: Calculates equivalent distances between conductors.

Generator and Load Subclasses

- Define physical & electrical parameters of loads and buses.
- Generator Class: Models a generator's electrical properties.
- Load Class: Models a load's electrical properties.

Class Specifications

This contains all of the classes, as well as specific definitions, outputs, and how to use them

Class Specifications

Bundle Class:

```
Class for determining the dsc and dsl of a bundle of conductors for a transmission line.
Parameters:
    name(str): name of the bundle
    num_conductors(int): number of conductors in bundle
    spacing(float): spacing between conductors
    conductor(Conductor): the conductor class to be used
```

Class Methods

calculate_dsc: Calculates the dsc value of the bundle based on the number of conductors, used for determining the admittance of the transmission line the bundle is a part of.

```
Returns:
    value(int): the dsc of the bundle
```

calculate_dsl: Calculates the dsl value of the bundle based on the number of conductors, used for determining the impedance of the transmission line the bundle is a part of.

```
Returns:
    value(int): the dsl of the bundle
```

Bus Class:

```
Class for defining a bus and the values relevant to it.
Parameters:
    name (str): name of the bus
    baseKV (float): base voltage of the bus
    vpu (float): Per unit bus voltage
    delta (float): Angle of the bus
    bus_type (str): Type of bus, used for injection and similar
```

Circuit Class:

```
Function for creating and manipulating circuit elements.
Parameters:
    name (string): Defines name of circuit.
    buses (dict): Dictionary for storing bus objects.
    transformers (dict): Dictionary for storing transformer objects.
    transmission_lines (dict): Dictionary for storing transmission lines.
    generators (dict): Dictionary for storing generators.
    loads (dict): Dictionary for storing loads.
```

Class Methods

add_bus: Creates a bus element and adds it to the buses dictionary, with its key being the name.

```
Parameters:
    name (string): Defines name of bus. Used for
    baseKV (float): Defines base KV.
    vpu (float): Defines vpu.
    delta (float): Defines delta.
    bus_type (str): Defines type of bus.
```

add_transformer: Creates a transformer element and adds it to the transformers dictionary, with its key being the name, bus1, and bus2.

```
Parameters:
    name (string): Defines name of transformer.
    bus1 (Bus): Defines first bus object.
    bus2 (Bus): Defines second bus object.
    power_rating (float): Defines power rating.
    impedance_percent (float): Defines impedance percent.
    x_over_r_ratio (float): Defines x_over_r_ratio.
    """
```

add_transmission_line: Creates a transmission line element and adds it to the transmission_lines dictionary, with its being the name, bus1, and bus2.

```
Parameters:
    name (string): Defines name of transmission line.
    bus1 (Bus): Defines first bus object.
    bus2 (Bus): Defines second bus object.
    bundle (Bundle): Defines bundle.
    geometry (Geometry): Defines geometry.
    length (float): Defines length.
```

add_generator: Creates a generator element and adds it to the generators dictionary, with its key being the name and the bus

```
Parameters:
    name (string): Defines name of generator.
    bus (Bus): Defines bus object.
    voltage_setpoint (float): Defines voltage set point.
    mw_setpoint (float): Defines mw set point.
```

add_load: Creates a load element and adds it to the loads dictionary, with its key being the name and the bus

```
Function for adding load elements.
Parameters:
    name (string): Defines name of load.
    bus (Bus): Defines bus object.
    real_power (float): Defines real power.
    reactive_power (float): Defines reactive power.
```

calc_ybus: Calculates the y bus matrix of the circuit function.

```
Returns:
    y_bus (pandas.core.frame.DataFrame): y bus matrix
```

The y bus matrix is a NxN matrix, with N being the number of Buses in the circuit, with the intersecting points being the admittances seen in the connection between the two buses. A sample output for the seven bus system used to test this is shown below:

	Bus1	Bus2	Bus3	Bus4	Bus5	Bus6	Bus7
Bus1	1.46329-14.63290j	-1.463290+ 14.632900j	0.000000+ 0.000000j	0.000000+ 0.000000j	0.000000+ 0.000000j	0.000000+ 0.000000j	0.000000+ 0.000000j
Bus2	-1.46329+14.63290j	38.506262-128.054052j	-10.583706+32.424808j	-26.459266+ 81.062019j	0.000000+ 0.000000j	0.000000+ 0.000000j	0.000000+ 0.000000j
Bus3	0.000000+ 0.000000j	-10.583706+ 32.424808j	23.813339-72.871379j	0.000000+ 0.000000j	-13.229633+ 40.531009j	0.000000+ 0.000000j	0.000000+ 0.000000j
Bus4	0.000000+ 0.000000j	-26.459266+ 81.062019j	0.000000+ 0.000000j	47.248689-144.631639j	-7.559790+ 23.160577j	-13.229633+ 40.531009j	0.000000+ 0.000000j
Bus5	0.000000+ 0.000000j	0.000000+ 0.000000j	-13.229633+40.531009j	-7.559790+ 23.160577j	47.248689-144.631639j	-26.459266+ 81.062019j	0.000000+ 0.000000j
Bus6	0.000000+ 0.000000j	0.000000+ 0.000000j	0.000000+ 0.000000j	-13.229633+ 40.531009j	-26.459266+ 81.062019j	41.270717-140.518560j	-1.581819+18.981824j
Bus7	0.000000+ 0.000000j	0.000000+ 0.000000j	0.000000+ 0.000000j	0.000000+ 0.000000j	0.000000+ 0.000000j	-1.581819+ 18.981824j	1.581819-18.981824j

The formatting does not look amazing here, due to the length of it when being generated, and creates the matrix using a Pandas Dataframe.

compute_power_injection: Computes the real and reactive power injection P & Q for all buses in the circuit

```
Parameters:
    busDict (dict): Dictionary for storing bus objects.
    yBusFrame (pandas.core.frame.DataFrame): y bus matrix
Returns:
    Px (pandas.core.frame.DataFrame): Real power injection (P) for all buses.
    Qx (pandas.core.frame.DataFrame): Real power injection (Q) for all buses.
```

This is done by through the power injection equations, which we obtained by having a nested loop through the busDict twice, getting a bus_k and bus_j each time, before setting V_k, delta_k, V_J, delta_j, and Y_kj using the bus objects, with Y_kj being the location in Y bus matrix. At the start of each K loop, before looping through J, we also set P_k or Q_k to 0 before looping through all J values. We then calculate the equations, with being the sum of all K and J values given by $V_k * V_j * \text{abs}(Y_{kj}) * \cos(\text{delta}_k - \text{delta}_j - \text{angle}(Y_{kj}))$. Q_k is calculated the same way, with the equation changed to be $V_k * V_j * \text{abs}(Y_{kj}) * \sin(\text{delta}_k - \text{delta}_j - \text{angle}(Y_{kj}))$. Once obtained, absolute the P_k and Q_k values are checked to be above the power tolerance, which is set to 10^{-10} in our code, and is set to 0 if less, before being stored into the output dataframes.

compute_power_mismatch: Computes the power mismatch (PM) for all buses.

```
Parameters:
    busDict (dict): Dictionary for storing bus objects.
    yBusFrame (pandas.core.frame.DataFrame): y bus matrix
Returns:
    power_mismatch (numpy.ndarray): Power mismatch (PM) for all buses, row 1
    being real and row 2 being imaginary.
```

This is done by first creating arrays to store all the required values in, before getting a list of the bus names from busDict and running the compute_power_injection function above. Once done, the method loops through all buses in the dict, before storing the Vpu and delta of the bus, and the injected real and reactive power of the bus from the compute_power_injection function. It then checks the type of bus, with Slack buses having no specified real or reactive power, PV buses only having specified real power, and PQ buses having both, with invalid bus types being set to 0, the same as if it was a slack bus. The specified values are just the real or reactive power of the bus divided by the Sbase, in our case 100MW. The injected powers are then subtracted from the specified ones, with both values being stored in the previously created arrays. Once all buses have been looped through, the arrays are concatenated and returned as output.

Conductor Class:

Defines a conductor in terms of the Aluminum Company of America Table A.4 for characteristics of ACSR.

Parameters:

name(str): name of the conductor
diameter(float): diameter of the conductor
GMR(float): GMR of the conductor
resistance(float): resistance of the conductor
ampacity(float): ampacity of the conductor

Generator Class:

Models a generator connected to a power system bus.

Parameters:

name (str): Generator identifier.
bus (Bus): Bus object the generator is connected to.
voltage_setpoint (float): Desired voltage in per unit (p.u.).
mw_setpoint (float): Real power generation in MW.

Geometry Class:

Represents the physical arrangement for conductors in a transmission line

Parameters:

name (str): Name of geometry config
xa, ya (float): Coordinates of conductor A
xb, yb (float): Coordinates of conductor B
xc, yc (float): Coordinates of conductor C

Class Methods:

calc_deq: Method that calculates the equivalent distance of the given geometry, used for finding impedance and admittance of the transmission line.

Returns:

Deq(float): Equivalent distance of geometry.

Load Class:

Models a load connected to a power system bus.

Parameters:

name (str): Load identifier.
bus (Bus): Bus object the load is connected to.
real_power (float): Real power demand in MW.
reactive_power (float): Reactive power demand in Mvar.

Settings Class:

```

Defines global values for use in other locations of the code
Parameters:
    frequency(float): Defines the frequency at which the code is based
    base_power(float): Defines the power at which the code is based in watts
    sbase(float): Defines the base power at which the code is based in MW

```

Values are set at Settings.frequency = 60, Settings.base_power = 100e6, Settings.Sbase = 100

Solution Class:

Solves the steady-state voltage magnitudes and angles across all buses using the Newton-Raphson method for power flow analysis.

In newton_raphson method:

Steps Performed:

1. Initializes voltage vector based on initial bus values.
2. Computes the power mismatch using the current voltages.
3. Reduces the mismatch vector by ignoring slack bus contributions.
4. Checks convergence against a specified mismatch tolerance.
5. Constructs the full Jacobian matrix using the **Jacobian** class.
6. Solves the linear system $J \cdot \Delta x = \text{mismatch}$ for voltage corrections.
7. Updates the voltage magnitudes and angles of all buses.
8. Repeats until mismatch is within tolerance or maximum iterations reached.

Returns:

- **voltages**: A NumPy array of complex bus voltages in per-unit after convergence.

SymFaultSolver Class:

Calculates the impact of a balanced (three-phase) fault at any bus using pre-fault conditions and the system's Thevenin equivalent impedance.

`circuit`: An instance of the `Circuit` class.

`prefault_voltages`: A NumPy array of the converged complex voltages at each bus, typically output from the Newton-Raphson method.

In `apply_fault`:

Steps Performed:

1. Converts the Ybus matrix to a Zbus matrix using matrix inversion.
2. Locates the diagonal element of Zbus (Z_{ii}) corresponding to the faulted bus.
3. Computes fault current using $I_{\text{fault}} = V_{\text{prefault}} / Z_{th}$
4. Calculates voltage drop across all buses due to the fault.
5. Returns updated bus voltages during the fault and the fault current.

Returns: A dictionary with:

- `"fault_bus"`: Name of the faulted bus.
- `"fault_current"`: The computed fault current in per-unit.
- `"voltage_during_fault"`: A NumPy array of bus voltages during the fault.

AsymFaultSolver Class:

Calculates the impact of unbalanced (SLG, LL, DLG) faults at any bus using pre-fault conditions and the system's Z_{pos} , Z_{neg} , and Z_{zero} impedance matrices.

`circuit`: An instance of the `Circuit` class.

prefault_voltages: A NumPy array of the converged complex voltages at each bus, typically output from the Newton-Raphson method.

fault_type: A string to determine which type of Asymmetric fault will be analyzed.

fault_impedance: A complex value to determine the impedance of the fault, set to be 0.

In apply_fault:

Steps Performed:

1. Gets the 3 Zbus matrices from the 3 Ybus matrices using matrix inversion.
2. Determine the type of fault to be analyzed, then solve for Z_{eq} of that fault type.
3. Computes fault currents using $V_{prefault}$ and Z_{eq} , and multiplier depending on fault type, 3 for SLG and $\sqrt{3}$ for LL
4. Find sequence voltages at all buses using fault current and Z Bus matrices.
5. Convert Sequence voltages to phase voltages using phase matrix
6. Return fault currents and phase voltages

Returns: A dictionary with:

- **"I_pos", "I_neg", "I_zero"**: Fault currents of each given Zbus type
- **"V_pos", "V_neg", "V_zero"**: Sequence voltages at faulted bus.
- **"V_phase"**: Phase voltages at faulted bus.
- **"V_phase_phasors"**: Phase voltages at faulted bus in phasor form
- **"V_phase_all"**: Matrix with all phase voltages at all buses

QUICK NOTICE: The SymFaultSolver and AsymFaultSolver are not outputting values equivalent to what the PowerWorld Simulator Provides. Use these at your discretion.

Transformer Class:

Represents a transformer in the power system using per-unit calculations.

Parameters:

name (str): Name of the transformer.
bus1, bus2 (Bus): Connected buses.
power_rating (float): Transformer rating in MVA.
impedance_percent (float): Transformer impedance in percentage.
x_over_r_ratio (float): Reactance-to-resistance ratio.
base_power (float): System base power in MVA (default: 100, pulled from settings file, with 1e-6 to compensate for settings being in VA not MVA).

Class Methods:**calc_impedance:**

Calculates the per-unit resistance (rpu) and reactance (xpu).

Parameters:

vbase_new (float, optional): Voltage base for per-unit calculation.
sbase_new (float, optional): Power base for per-unit calculation.

Returns:

tuple: (rpu, xpu) representing resistance and reactance in per-unit.

calc_admittance: Takes inverse of impedance to get admittance

Computes per-unit admittance (yseries).

Returns:

float: Admittance value in per-unit

calc_y_primitive:

Computes the primitive admittance matrix (y_primitive) and formats it using pandas.

Returns:

DataFrame: 2x2 primitive admittance matrix with bus names as labels.

Dataframe array is in form of `[[calc_admittance, -calc_admittance], [-calc_admittance, calc_admittance]]`

TransmissionLine Class:

Creates and models the necessary values of a transmission line with the purpose of creating a primitive y bus matrix

Parameters:

name(str): name of the transmission line
bus1(Bus): bus object, used for determining Vbase
bus2(Bus): bus object
bundle(Bundle): bundle object
geometry(Geometry): geometry object

```
length(float): length of the transmission line
```

Class Methods:

calculate_impedance: Calculates the impedance using references to geometry deq, bundle dsl, bundle num_conductors, and conductor resistance, with base equation $w \cdot 2 \cdot 10^{-7} \cdot \log(\text{DEQ}/\text{DSL})$, $w = 2 \cdot \pi \cdot \text{frequency}$

```
Returns:  
float: impedance of the transmission line
```

calculate_admittance: Calculates the admittance of the transmission line, using the base equation $w \cdot 2 \cdot \pi \cdot e_0 / \log(\text{DEQ}/\text{DSC})$ $w = 2 \cdot \pi \cdot \text{frequency}$

```
Returns:  
float: admittance of the transmission line
```

calculate_admittance_matrix: Calculates the primitive admittance matrix, using full values for the admittance and impedance.

```
Returns:  
end_matrix(pd.dataframe): primitive admittance matrix
```

calculate_y_primitive: Calculates the Y primitive matrix for use in other functions, same as above except using per unit values instead of full values.

```
Returns:  
end_matrix(pd.dataframe): y primitive matrix
```

Main Program Execution:

The `main.py` script serves as the entry point for setting up and simulating the 7-bus power system using the advanced Python-based simulator. It demonstrates how to initialize the system, define components, run power flow analysis, validate matrices, and simulate symmetrical faults.

Key Functional Steps:

1. System Initialization:

- Creates a new `Circuit` object with a 100 MVA base.
- Adds seven buses, including a Slack bus, PQ buses, and a PV generator bus.

2. Component Setup:

- Defines a standard ACSR conductor ("Partridge") and constructs a transmission **Bundle** and **Geometry** layout for line modeling.
- Adds six transmission lines with specified connections and lengths.
- Adds two two-winding transformers (125 MVA and 200 MVA ratings).
- Specifies loads on Bus 3, 4, and 5.
- Connects a 200 MW generator to Bus 7 (PV bus).

3. **Per-Unit Parameter Display:**

- Calculates and prints per-unit resistance (R), reactance (X), and susceptance (B) for all transformers and transmission lines.

4. **Ybus Matrix Computation:**

- Computes and displays the full admittance matrix (Ybus) of the system using the Pandas DataFrame format.

5. **Jacobian Matrix Validation:**

- Uses initial bus voltage estimates to calculate the Jacobian matrix.
- Outputs the Jacobian shape and contents for inspection and comparison.

6. **Power Flow Analysis (Newton-Raphson):**

- Calls the **Solution** class to solve the nonlinear power flow problem using the Newton-Raphson method.
- Prints bus voltages and angles after each iteration and at convergence.

7. **Symmetrical Fault Simulation:**

- After obtaining converged pre-fault voltages, apply a 3-phase fault at Bus 3 using the **SymFaultSolver** class if circuit is in Fault solver mode and fault type is **SYM**

- Outputs the fault current and post-fault voltages at all buses.

8. **Asymmetrical Fault Simulation:**

- After obtaining converged pre-fault voltages, apply an asymmetrical fault at Bus 3 using the `AsymFaultSolver` class if circuit is in Fault solver mode and fault type is NOT `SYM`
- Outputs the phase voltages at each bus and the faulted currents.

9. **Final Output Summary:**

- Repeats a Newton-Raphson power flow with limited iterations to showcase voltage behavior.
- Displays final voltage magnitudes and phase angles in per-unit form.

This project successfully developed a fully functional advanced power system simulator capable of modeling a 7-bus looped transmission network. The simulator incorporates key features of modern power flow analysis, including the Newton-Raphson iterative method and symmetrical three-phase fault simulation.

By modularly designing the system through clearly defined Python classes — such as `Circuit`, `Bus`, `Transformer`, `TransmissionLine`, `Generator`, and `Load` — the simulator allows for flexible and expandable power system modeling. Each component was built with attention to physical parameters like impedance, admittance, and per-unit system consistency, ensuring realistic simulation behavior.

The simulator was rigorously validated by:

- Calculating the Ybus matrix and ensuring correct network admittance formation,
- Constructing and verifying the Jacobian matrix for Newton-Raphson iterations,
- Running full power flow analysis to convergence within reasonable tolerances,

- Simulating a three-phase fault at a specified bus and computing post-fault voltages and currents.

The structure of the code also allows future expansion, such as incorporating different load models or interfacing with larger network models. Overall, the project demonstrates a deep integration of power system theory with practical computational methods, highlighting the critical skills necessary for modern power system analysis.